

SPEC class

A simple interface to access device parameters in test programs

Helmut Osterloher (LP DCDC Converter)

SPEC class - Overview

- **Provides structured access to device parameters in test programs**
- **Parameters are stored in one place (external file)**
 - Single source of information
 - Easy to read syntax including units
 - Easy to maintain
- **No program coding and re-compiling/re-starting necessary when adding a parameter**

example.spec

[DEFAULT]

```
; Parameters common to all device versions
Vout          = 5.0 V           ; this is a comment
Vref          = 200 mV
Vin_min       = 2.3 V
Vin_max       = 5.5 V
; Lookup table format is a comma separated list of: <integer>:<double>[<unit>]
VLDO1         = 0:3.0V,1:3.1V,2:3.2V,4:3.4V
; For better overview the following syntax is also supported
VLDO2         = 0:3.0V,
               1:3.1V,
               2:3.2V,
               4:3.4V
```

[TPS61240]

```
; Parameters specific for a certain device version
Vout          = 4.2 V
; The parameter value can be specified depending on TI_ShellGetTempState() (e.g. for trim targets)
; Temperature state format is a comma separated list of: <amb|hot|cold>:<double>[<unit>]
Vref          = amb:200 mV,
               hot:204 mV
has_PG        = true           ; has Power Good feature
```

[TPS61241]

```
Vout          = 5.0 V
Vref          = amb:1.229 V,
               hot:1.234 V
has_PG        = false         ; Power Good not implemented
```

General

- The file format uses sections and key/value pairs like *.ini files
- The [DEFAULT] section contains parameters common to all device versions/spins
- The [<device_name>] section contains parameters for a particular spin.
- Lookup order
 1. Search in specific section e.g. [TPS61241]
 2. Search in [DEFAULT]

What do I need to add to the test program?

- Add **spec** and **inireader** library from SVN to your project
- Add spec global object to the main application file (e.g. ef2640.cpp)
SPEC spec;
- Add **#include "spec/spec.hxx"** to the application header file (e.g. ef2640.h)
- Add extern statement to the application header file
extern SPEC spec;
- Call initialization in OnTestInit():
spec.init("TPS6125X.spec");
- Select device version (typically also in OnTestInit()):
spec.select_device("TPS61240");

Get Parameters

- Get value for selected device

```
double value = spec("Parameter Name");  
const char *str = spec("Parameter Name").get_str();
```

- Get value for specific device version

```
double value = spec["Device Name"]("Parameter Name");  
const char *str = spec["Device Name"]("Parameter Name").get_str();
```

Set Parameters (at run-time)

- Setup spec global object to **unprotect** all parameters from accidental overwrite

```
SPEC spec(true) ;
```

- **Or** use the following methods to unprotect/protect a single parameter

```
spec("Parameter Name").unprotect() ; // allow overwrite
```

```
spec("Parameter Name").protect() ; // disallow overwrite
```

- Set value for selected device

```
spec("Parameter Name") = 3.6;
```

- Set value for specific device version

```
spec["Device Name"]("Parameter Name") = 3.6;
```

Lookup Tables

- Lookup table format is a comma separated list of:
<integer>:<double>[<unit>]

- Example:

```
VLD02 = 0:3.0V,  
        1:3.1V,  
        2:3.2V,  
        4:3.4V
```

- Get value for <index> from lookup table

```
double value = spec("Lookup Table Name") [<index>] ;
```

Units & Scaling

- The following unit prefixes and units are supported

M	1e+6
k	1e+3
%	1e-2
m	1e-3
u	1e-6
n	1e-9
p	1e-12

V	Volt
A	Ampere
Ohm	Ohm
s	Second
Hz	Hertz
H	Henry
F	Farad
W	Watt
C or DGR	Degree Celsius

Temperature dependent parameters

- The parameter value can be specified depending on `TI_ShellGetTempState()` (e.g. for trim targets) with the following syntax:

<amb|hot|cold>:<double>[<unit>]

- Example

**Vref = amb:200 mV,
 hot:204 mV**

Arithmetic operations

+	add / unary plus
-	subtract / unary minus
*	multiply
/	divide
&	bitwise AND ¹⁾
	bitwise OR ¹⁾
<<	bit shift left ¹⁾
>>	bit shift right ¹⁾
^	bitwise XOR ¹⁾

[DEFAULT]

[TPS61252]

```
vref_target      = 1.2 V
pg_th_rising     = 95% * vref_target
pg_th_falling    = 90% * vref_target
```

¹⁾ Note: all bitwise operations are internally using **unsigned int** (32 bit) type casts

Data types in *.spec

204.0 mV	double
0xFA01	hexadecimal (32 bits max)
b10111001	binary (32 bits max)
true / false	Boolean
2.5E3 or 2.3e-3	scientific notation
"min"	string

Note1: all data types (except strings) will be returned as **double** when accessed at run-time

Note2: strings can be accessed with **spec("Parameter Name").get_str()**

Questions?